

Analysis of Runs of Homozygosity in *Acropora cervicornis*

Trinity Conn

2025-10-28

An analysis of population risk, genomic erosion, and recent demographic history using runs of homozygosity

Runs of Homozygosity (ROH) are regions of the genome in an individual of extensive homozygosity. ROH develop from individuals sharing the same haplotype due to IBD or IBS. Runs of homozygosity are typically broken down through recombination and reproduction with less genetically similar individuals. Therefore, extremely long blocks are typically produced through recent inbreeding. ROH are of particular conservation concern because in populations that have experienced rapid population decline, inbreeding is more likely. Long blocks of homozygous loci risk the expression of deleterious loci that have been masked in the heterozygous state previously.

Identifying ROH using Garlic (Szpiech *et al* 2019)

```
#!/bin/bash

#set up environment
source ~/.bashrc
conda activate garlic

cd /scratch1/tlc_975

#define variables
#I used plink to create the necessary tfam/tped files
#the cent2file is the description of the location of centromeres as defined by centrominer.
TPED="acerroh.tped"
TFAM="acerroh.tfam"
CENT="cent2.file"

#run garlic
/home1/tlc_975/linux/garlic --tped $TPED --tfam $TFAM --centromere $CENT --error 0.001 --auto-winsize --

##load in required packages
library(rtracklayer)

## Loading required package: GenomicRanges

## Loading required package: stats4
```

```

## Loading required package: BiocGenerics

##
## Attaching package: 'BiocGenerics'

## The following objects are masked from 'package:stats':
##
##     IQR, mad, sd, var, xtabs

## The following objects are masked from 'package:base':
##
##     anyDuplicated, aperm, append, as.data.frame, basename, cbind,
##     colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
##     get, grep, grepl, intersect, is.unsorted, lapply, Map, mapply,
##     match, mget, order, paste, pmax, pmax.int, pmin, pmin.int,
##     Position, rank, rbind, Reduce, rownames, sapply, saveRDS, setdiff,
##     table, tapply, union, unique, unsplit, which.max, which.min

## Loading required package: S4Vectors

##
## Attaching package: 'S4Vectors'

## The following object is masked from 'package:utils':
##
##     findMatches

## The following objects are masked from 'package:base':
##
##     expand.grid, I, unname

## Loading required package: IRanges

## Loading required package: GenomeInfoDb

library(GenomicRanges)
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.2
## v ggplot2    4.0.0      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0

## -- Conflicts ----- tidyverse_conflicts() --
## x lubridate::%within%() masks IRanges::%within%()
## x dplyr::collapse()     masks IRanges::collapse()
## x dplyr::combine()      masks BiocGenerics::combine()
## x dplyr::desc()         masks IRanges::desc()
## x tidyr::expand()       masks S4Vectors::expand()

```

```
## x dplyr::filter()      masks stats::filter()
## x dplyr::first()      masks S4Vectors::first()
## x dplyr::lag()        masks stats::lag()
## x ggplot2::Position() masks BiocGenerics::Position(), base::Position()
## x purrr::reduce()     masks GenomicRanges::reduce(), IRanges::reduce()
## x dplyr::rename()     masks S4Vectors::rename()
## x lubridate::second() masks S4Vectors::second()
## x lubridate::second<-() masks S4Vectors::second<-()
## x dplyr::slice()      masks IRanges::slice()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(plyranges)
```

```
##
## Attaching package: 'plyranges'
##
## The following objects are masked from 'package:dplyr':
##
##   between, n, n_distinct
##
## The following object is masked from 'package:IRanges':
##
##   slice
##
## The following object is masked from 'package:stats':
##
##   filter
```

```
library(karyoploteR)
```

```
## Loading required package: regioneR
```

```
library(cowplot)
```

```
##
## Attaching package: 'cowplot'
##
## The following object is masked from 'package:lubridate':
##
##   stamp
```

```
library(geneviewer)
```

```
##
## Attaching package: 'geneviewer'
##
## The following objects are masked from 'package:plyranges':
##
##   read_bed, read_gff
```

```

library(RColorBrewer)

#generate colors based on Paige's Figures
custom_colors<-c("Aruba/Curacao"= "#04A2A0", "Belize/MX"="#AA36A5", "FL"="#EE962F", "JAM/DR"="#C6CF18",

##load in bed file -- this is the output of the regions of runs of hom from garlic, and I have modified
#"awk '
#BEGIN { track_line = "" }
#/~track/ { track_line = $0; next }
#/~browser/ { next }
#{ print $0 "\t" track_line }
#' acerroh.roh.bed > acerroh.track.bed"

#This has added in the individual from the tracks so i can identify the populations.

#setwd("~/Documents/Shedd/acer_analysis")

acer<-geneviewer::read_bed("../data/acerroh.track.bed")

## Warning in (track_line + 1):length(lines): numerical expression has 143653
## elements: only the first used

#add in indiv column
df<-acer%>%
  mutate(indv = str_extract(blockCount
                             , '(<=Ind: ) [A-Za-z0-9_-]+'))

##load in population metadata
pop<-read.table("../data/popfile_2.txt", header=TRUE)

#combine in population data

df2<-merge(df, pop, by=c("indv"))
df2<-df2%>%
  select("chrom", "indv", "pop", "chromStart", "chromEnd", "name", "score")

df3<-df2%>%
  filter(score>=10000)

```

Timing ROH

What is a 'long' ROH in coral?

Previous studies assessing ROH in at-risk populations have focused on mostly terrestrial animals, wolves, birds, and some marine species such as the vaquita. All of these species have genomes on the order of 4-6x the size of the caribbean Acroporid genome size. Therefore, previous estimates of what constitutes 'long' ROH in these species has been typically limited to >1tb in length. In *Acropora cervicornis*, no ROH exceeds 1 tb in length, so it seems as though this species has no particularly long runs of homozygosity. However we can use recombination rate and the genome size to calculate what is an effectively long ROH in *Acropora cervicornis*. In the following code chunk we calculate the recombination rate using the previously generated linkage map in Locatelli et al. 2023 that has been adapted to the Sanger-generated genome used in this paper.

Following that, we calculate the minimum length of a run of homozygosity in *Acropora cervicornis* to have been generated 25 generations ago. Theoretically, we expect that strongly deleterious alleles (alleles that would cause inbreeding depression if harbored in the homozygous state in the population) to be selected against so strongly that they would be lost in only a few generations, likely no more than 25. Therefore to make sure we are only assessing runs that are likely to contain deleterious alleles within them, we can use the recombination rate to ‘age’ the ROH.

```
#calculate recombination rate using Nick's linkage map applied to the sanger genome

map<-read.table("../data/Acervicornis_SEXAVG_sanger.map")
colnames(map)[1]<-"chrom"
colnames(map)[2]<-"snpid"
colnames(map)[3]<-"genomic position (cM)"
colnames(map)[4]<-"snp position (bp)"

#calculate recombination rate in cM/Mb

map<-map%>%
  filter(`snp position (bp)`!=0)%>%
  mutate(rate=`genomic position (cM)`/(`snp position (bp)`)

mapsum<-map%>%
  group_by(chrom)%>%
  summarize(mean=mean(rate))

#translate ROH length in bp to length in cm

#score is the length of the ROH in bp, if you have x cm / bp to get position in centimorgans
#multiply the length (score) by the mean recombination rate per chromosome

mapsum$chrom<-paste0("chr",mapsum$chrom)
df_recomb<-full_join(mapsum, df3, by=c("chrom"))

df_recomb<-df_recomb%>%
  # mutate(scoretest=10000000*score)
df_recomb<-df_recomb%>%
  mutate(cM=score*mean)

#estimate generations

df_recomb<-df_recomb%>%
  mutate(g=(100/(2*cM)))

#years, generation time 5 years?

df_recomb<-df_recomb%>%
  mutate(yr=g*5)

#identifying min ROH length for 25 generations or more recent in this coral species
gen25<-df_recomb%>%
  filter(g<=25)%>%
  summarise(min(score))
gen25
```

```
## # A tibble: 1 x 1
##   'min(score)'
##       <int>
## 1       228944
```

```
#identify max number of generations ago as LCA for ROH with a min length of 100kb
gen100<-df_recomb%>%
  filter(score>=100000)%>%
  summarise(max(g))
gen100
```

```
## # A tibble: 1 x 1
##   'max(g)'
##       <dbl>
## 1       193.
```

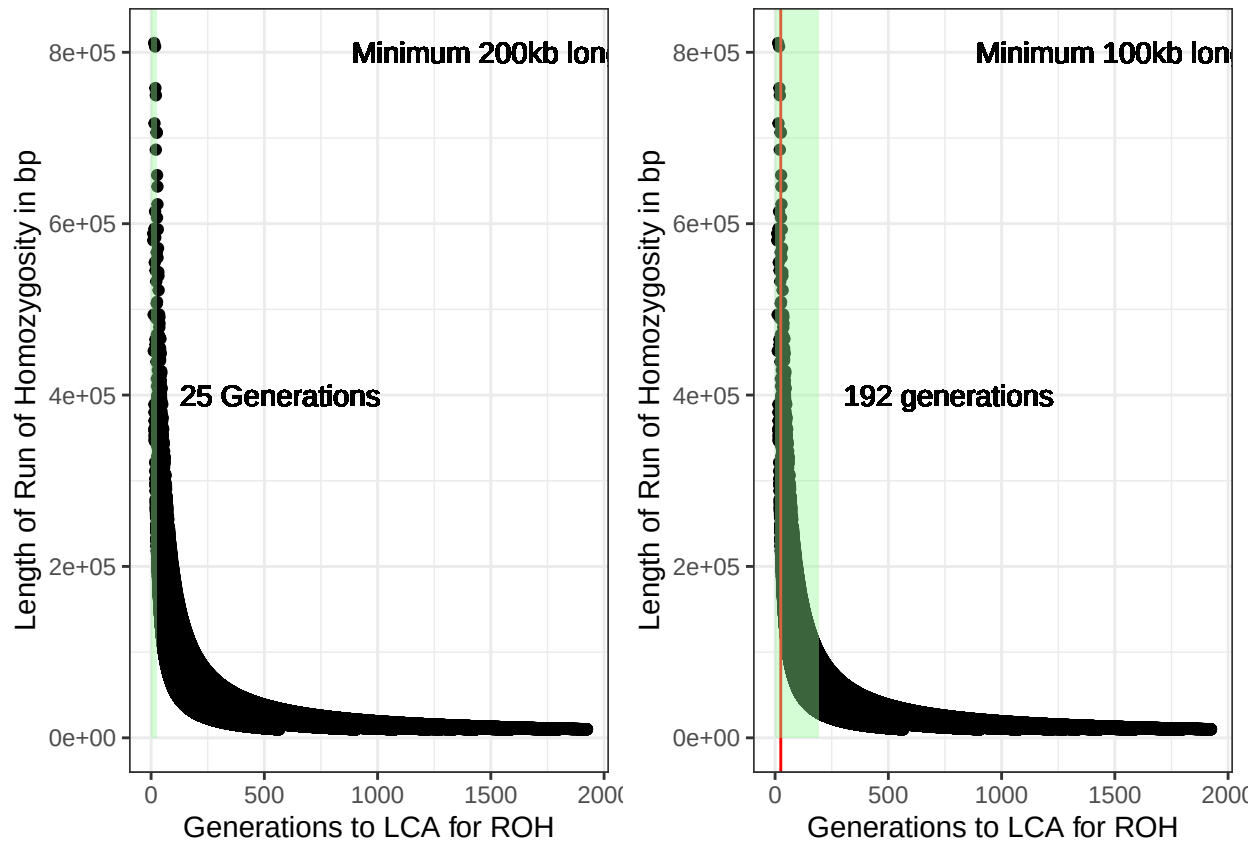
```
g200<-ggplot(df_recomb, aes(x=g, y=score))+geom_point()+theme_bw()+xlab("Generations to LCA for ROH")+ylab("Score")
g100<-ggplot(df_recomb, aes(x=g, y=score))+geom_point()+theme_bw()+geom_vline(xintercept=25, color="red")+
  #plotgrid
cowplot::plot_grid(g200, g100)
```

```
## Warning in geom_text(aes(40, 4e+05, label = "25 Generations", hjust = -0.1)): All aesthetics have length 1
## i Please consider using 'annotate()' or provide this layer with data containing a single row.
```

```
## Warning in geom_text(aes(1500, 8e+05, label = "Minimum 200kb long Runs", : All aesthetics have length 1
## i Please consider using 'annotate()' or provide this layer with data containing a single row.
```

```
## Warning in geom_text(aes(210, 4e+05, label = "192 generations", hjust = -0.1)): All aesthetics have length 1
## i Please consider using 'annotate()' or provide this layer with data containing a single row.
```

```
## Warning in geom_text(aes(1500, 8e+05, label = "Minimum 100kb long Runs", : All aesthetics have length 1
## i Please consider using 'annotate()' or provide this layer with data containing a single row.
```



#The appropriately 'long' ROH is <25 generations

#the length of ROH at 25 generations according to the paper (Kyriazis et al 2025) is 228944 (~200 kb) -

What is a 'long' ROH in coral?

Based on the calculated recombination rate, the minimum length of ROH at 25 generations is 228944 bp (~200 kb). Therefore we are going to analyze produced two datasets: one where only ROHs that are a minimum of 200kb are kept, and one where only ROHs of a minimum of 100kb are kept. ROHs that are at least 100kb in length are maximum 192 generations from the LCA. While this is unlikely to still contain deleterious alleles, In other species with much longer ROHs, runs of 100kb still contained deleterious variation, so we will continue to also look at runs of 100kb.

non-ROH heterozygosity

To calculate the IDRisk score, we need to also calculate heterozygosity when not accounting for the ROH blocks we have identified. To do this I removed all regions contained in the blocks (200kb and 100kb) from the vcf, and then calculated per-individual heterozygosity using vcftools. non-ROH heterozygosity is a metric that will provide insight into inbreeding load. Inbreeding load is the amount of lethal or deleterious loci that are currently masked in the heterozygous state –aka how much is inbreeding going to bring deleterious alleles into the homozygous state. This is not a perfect metric of inbreeding load, it is stating that the greater heterozygosity outside of these blocks, the greater risk that there are lethal alleles being masked, however in the context of an at-risk population (where we may expect more inbreeding to occur) it may serve as a reasonable metric of population risk.

```
#!/bin/bash
#Now we can calculate the non-ROH heterozygosity

cd scratch1/tlc_975

#filter the roh bed files to just runs longer than 200kb and 100 kb
awk '($3 - $2) >= 200000' acerroh.roh.bed > acerroh.roh200kb.bed
awk '($3 - $2) >= 100000' acerroh.roh.bed > acerroh.roh100kb.bed

#generate a vcf with just the sites not within the runs of homozygosity
bedtools intersect -a /project2/ckenkel_26/Acer_WGS/vcfs/Sep2025_morePAN/full.panel_5bpind.morePAN.HWE.1
bedtools intersect -a /project2/ckenkel_26/Acer_WGS/vcfs/Sep2025_morePAN/full.panel_5bpind.morePAN.HWE.1

#calculate heterozygosity
vcftools --vcf full.panel_nonroh100kb.vcf --het --out full.panel100kb.het
vcftools --vcf full.panel_nonroh200kb.vcf --het --out full.panel200kb.het
```

Calculate Population Risk

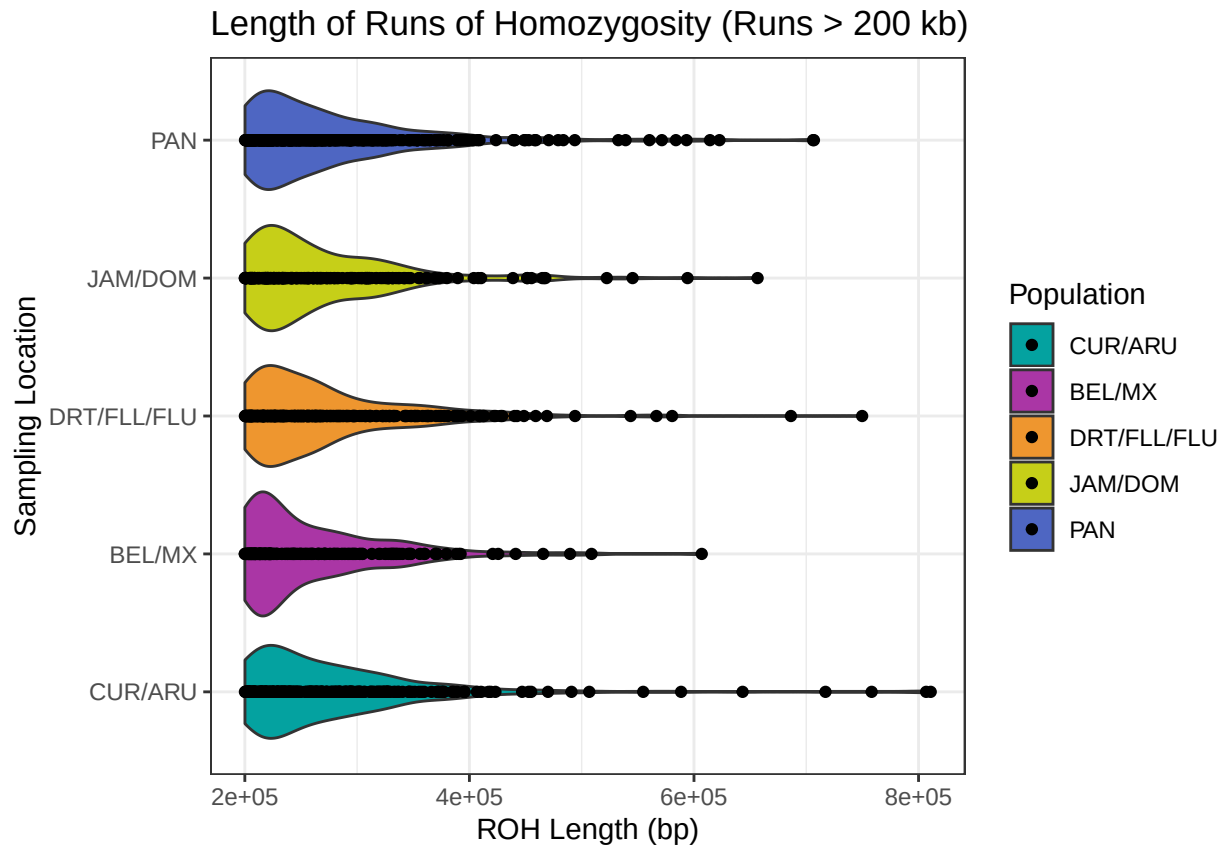
Now we will calculate the IDrisk score introduced in Kyriazis *et al*, 2025, which is the product of the Fraction of ROH across the genome (recent inbreeding) and the non-ROH heterozygosity (future inbreeding load). We do this for both a minimum ROH length of 100kb and a minimum ROH length of 200kb. **IDrisk = FROH * non-ROH Heterozygosity**

```
#setwd("~/Documents/Shedd/acer_analysis")
#Try and calculate
#the non-ROH heterozygosity values were calculated by filtering the vcf by removing regions in the roh

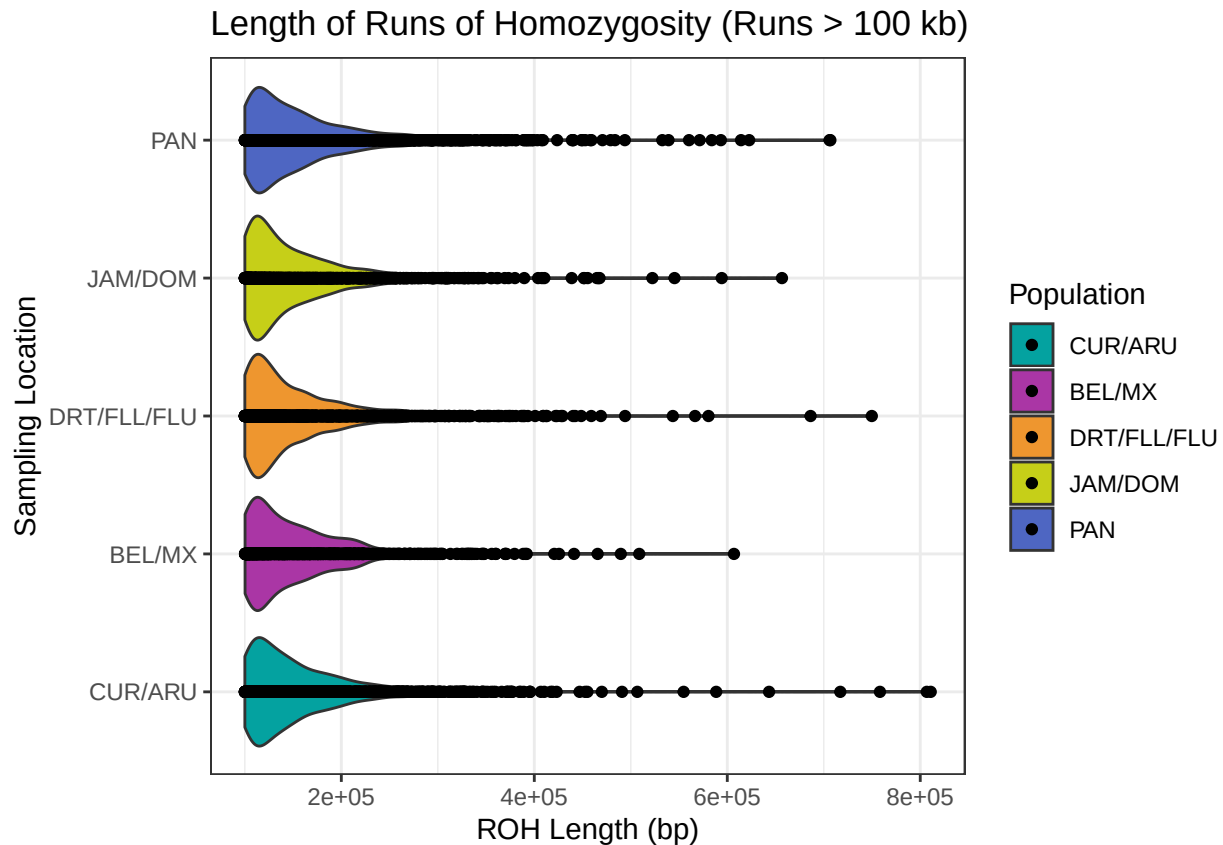
#calculate FROH 200 kb & 100 kb
#genome size of Acer
genomesize<-328680610
genomesizekb<-genomesize/1000

#filter out any ROH shorter than 100kb
df3100<-df2%>%
  filter(score>=100000)
#filter out any ROH shorter than 200kb
df3200<-df2%>%
  filter(score>=200000)

#plot the distribution of run length
ggplot(df3200, aes(x=score, y=pop, fill=pop))+geom_violin()+geom_point()+xlab("ROH Length (bp)") +
  ggtitle("Length of Runs of Homozygosity (Runs > 200 kb)") + ylab("Sampling Location") + scale_fill_manual
```



```
ggplot(df3100, aes(x=score, y=pop, fill=pop))+geom_violin()+geom_point()+xlab("ROH Length (bp)") +
  ggtitle("Length of Runs of Homozygosity (Runs > 100 kb)") + ylab("Sampling Location") + scale_fill_manual
```



```
#calculate FROH (fraction of the genome that is contained within runs of homozygosity)
```

```
#for 100kb
```

```
FROH100<-df3100%>%
  group_by(indv)%>%
  mutate(totalRoh=sum(score))%>%
  mutate(FROH=totalRoh/genomesize)%>%
  distinct(indv, .keep_all = TRUE)%>%
  dplyr::select(c('indv', 'pop', 'totalRoh', 'FROH'))
FROH100$limit_kb<-'100'
head(FROH100)
```

```
## # A tibble: 6 x 5
## # Groups:   indv [6]
##   indv      pop      totalRoh  FROH limit_kb
##   <chr>    <chr>        <int>  <dbl> <chr>
## 1 AR_113  Aruba/Curacao 28325217 0.0862 100
## 2 AR_115  Aruba/Curacao 20747878 0.0631 100
## 3 AR_117  Aruba/Curacao 30987059 0.0943 100
## 4 AR_119  Aruba/Curacao 29105389 0.0886 100
## 5 CRF_Acer-059 FL      21087246 0.0642 100
## 6 CRF_Acer-099 FL      16313338 0.0496 100
```

```
#for 200kb
```

```
FROH200<-df3200%>%
  group_by(indv)%>%
```

```

mutate(totalRoh=sum(score))>%
mutate(FROH=totalRoh/genomesize)>%
distinct(indv, .keep_all = TRUE)>%
dplyr::select(c('indv', 'pop', 'totalRoh', 'FROH'))
FROH200$limit_kb<-'200'
#FROH<-rbind(FROH100,FROH200)
head(FROH200)

```

```

## # A tibble: 6 x 5
## # Groups:   indv [6]
##   indv      pop      totalRoh  FROH limit_kb
##   <chr>    <chr>      <int>  <dbl> <chr>
## 1 AR_113  Aruba/Curacao 10677792 0.0325 200
## 2 AR_115  Aruba/Curacao  4652817 0.0142 200
## 3 AR_117  Aruba/Curacao 13119606 0.0399 200
## 4 AR_119  Aruba/Curacao  6979044 0.0212 200
## 5 CRF_Acer-059 FL          5859871 0.0178 200
## 6 CRF_Acer-099 FL          3837097 0.0117 200

```

```

#calculate non ROH heterozygosity
nonroh100<-read.table("../data/full.panel100kb.het.het", header=TRUE)
nonroh200<-read.table("../data/full.panel200kb.het.het", header=TRUE)

```

```

colnames(nonroh100)[1]<-"indv"
colnames(nonroh200)[1]<-"indv"
roh100<-merge(nonroh100,pop, by=c('indv'))
roh200<-merge(nonroh200,pop, by=c('indv'))

```

```

roh100<-roh100>%
  mutate(nsiteskb=N_SITES/1000)

```

```

#calculate nsites het per kb of genome size
roh100<-roh100>%
  mutate(heto=(N_SITES-O.HOM.)/genomesizekb)

```

```

roh200<-roh200>%
  mutate(nsiteskb=N_SITES/1000)
roh200<-roh200>%
  mutate(heto=(N_SITES-O.HOM.)/genomesizekb)

```

```

#calculate IDRisk score -- a sum of the fraction of genome within a run of homozygosity and
#IDrisk min 100kb
idrisk100<-merge(roh100, FROH100, by=c("indv"))
idrisk100<-idrisk100>%
  mutate(idrisk=heto*FROH)
#IDrisk min 200kb
idrisk200<-merge(roh200, FROH200, by=c("indv"))
idrisk200<-idrisk200>%
  mutate(idrisk=heto*FROH)
head(idrisk100)

```

```

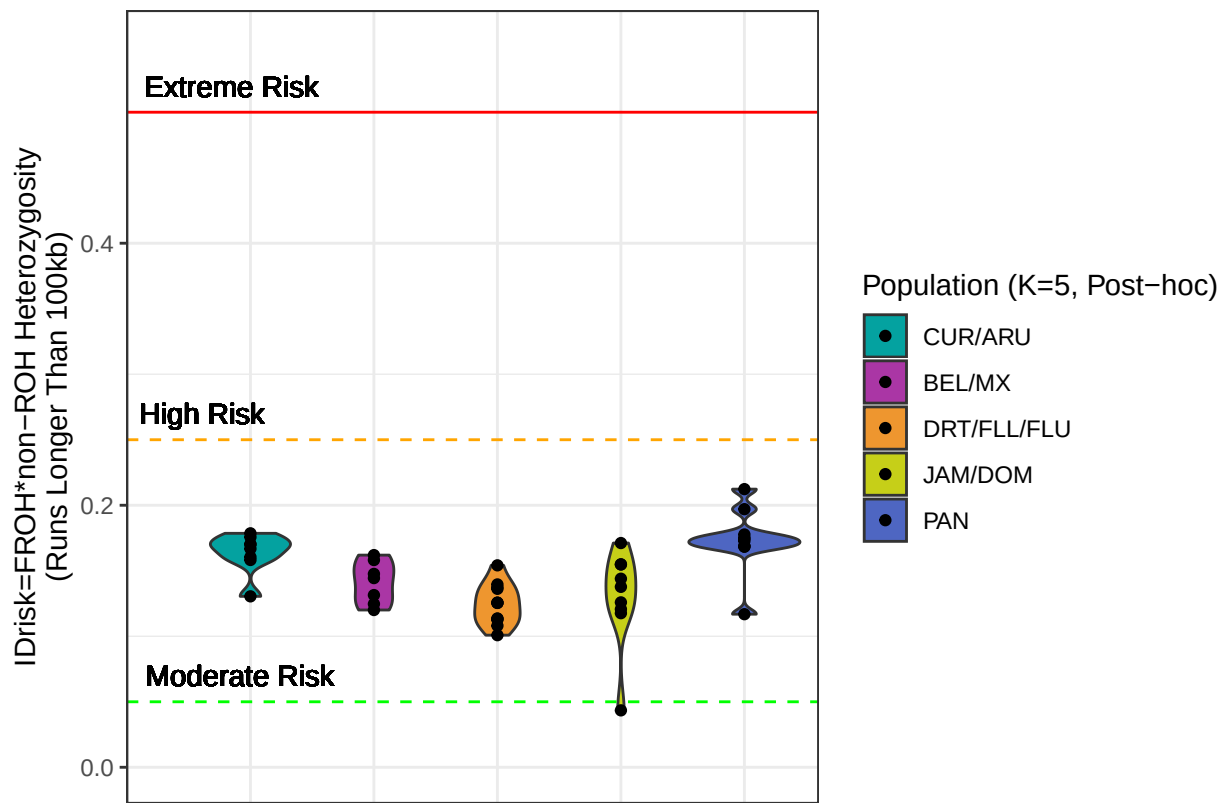
##           indv  O.HOM.  E.HOM.  N_SITES      F      pop.x nsiteskb      heto

```

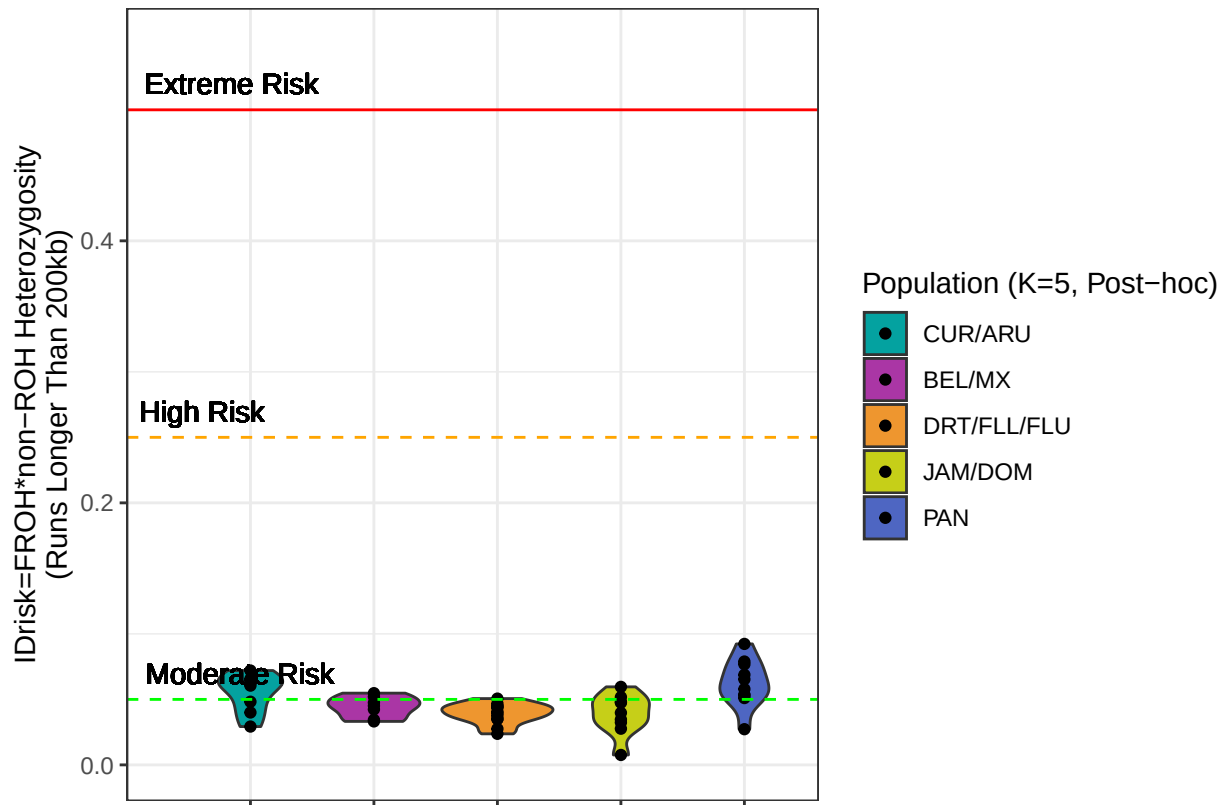
```
## 1      AR_113 1834154 1771970 2446257 0.09222 Aruba/Curacao 2446.257 1.862303
## 2      AR_115 1767905 1773180 2447232 -0.00783 Aruba/Curacao 2447.232 2.066830
## 3      AR_117 1853173 1772654 2446246 0.11954 Aruba/Curacao 2446.246 1.804405
## 4      AR_119 1820072 1767407 2439237 0.07839 Aruba/Curacao 2439.237 1.883789
## 5 CRF_Acer-059 1824618 1787947 2468282 0.05390                FL 2468.282 1.958327
## 6 CRF_Acer-099 1794400 1783896 2462139 0.01549                FL 2462.139 2.031574
##      pop.y totalRoh      FROH limit_kb      idrisk
## 1 Aruba/Curacao 28325217 0.08617855      100 0.1604906
## 2 Aruba/Curacao 20747878 0.06312474      100 0.1304681
## 3 Aruba/Curacao 30987059 0.09427711      100 0.1701141
## 4 Aruba/Curacao 29105389 0.08855219      100 0.1668137
## 5                FL 21087246 0.06415726      100 0.1256409
## 6                FL 16313338 0.04963280      100 0.1008327
```

```
#plot with risk evaluations
```

```
ggplot(idrisk100, aes(x=pop.x,y=idrisk, fill=pop.x))+geom_violin()+geom_point()+theme_bw()+
  geom_hline(yintercept = 0.25, color = "orange", linetype = "dashed")+
  geom_hline(yintercept=0.5, color='red', linetype='solid')+geom_hline(yintercept=0.05, color='green',
  ylim(0,0.55))+ylab("IDrisk=FROH*non-ROH Heterozygosity\n(Runs Longer Than 100kb)")+xlab("")+scale_fill(
  theme(axis.text.x = element_blank())+geom_text(aes(0,0.27,label = 'High Risk',hjust=-0.1))+geom_text(aes(0,0.55,label = 'Extreme Risk',hjust=-0.1))+geom_text(aes(0,0.05,label = 'Moderate Risk',hjust=-0.1))
```



```
ggplot(idrisk200, aes(x=pop.x,y=idrisk, fill=pop.x))+geom_violin()+geom_point()+theme_bw()+
  geom_hline(yintercept = 0.25, color = "orange", linetype = "dashed")+
  geom_hline(yintercept=0.5, color='red', linetype='solid')+geom_hline(yintercept=0.05, color='green',
  ylim(0,0.55))+ylab("IDrisk=FROH*non-ROH Heterozygosity\n(Runs Longer Than 200kb)")+xlab("")+scale_fill(
  theme(axis.text.x = element_blank())+geom_text(aes(0,0.27,label = 'High Risk',hjust=-0.1))+geom_text(aes(0,0.55,label = 'Extreme Risk',hjust=-0.1))+geom_text(aes(0,0.05,label = 'Moderate Risk',hjust=-0.1))
```



```
#ggplot(average, aes(x=meanFROH, y=meanhet, color=pop.x))+geom_point(size=3)+
  #scale_color_manual(values=custom_colors, labels=c("CUR/ARU", "BEL/MX", "DRT/FLL/FLU", "JAM/DOM", "#P
  # theme_bw()+scale_x_discrete(labels=c("CUR/ARU", "BEL/MX", "DRT/FLL/FLU", "JAM/DOM", "PAN"))
```

Visualizing Shared Runs of Homozygosity across Populations and Individuals

Finally, we can see if there are any large runs of homozygosity across the chromosomes that may be shared. This could indicate these individuals all come from recent inbreeding of the same individuals and already carry similar ROH, or identify where in the chromosome these long blocks are propagated in the population and if there are certain genes in that region. Below we look just across chromosome 4, but this can be plotted across all 4 genomes and plotted alongside other metrics of demographic history and population fitness.

```
#plot some of the genomic ranges to see if there are shared runs across populations

#subset just to chromosome 4 for now
roh_plot <- df3200 %>%
  mutate(length = chromEnd - chromStart) %>%
  arrange(pop, indv) %>%
  mutate(indv_id = as.numeric(factor(indv, levels = unique(indv)))) %>%
  filter(chrom=="chr0Z035969.1")

ggplot(roh_plot, aes(xmin = chromStart/1e6, xmax = chromEnd/1e6,
  ymin = indv_id - 0.4,
  ymax = indv_id + 0.4,
  fill = pop)) +
  geom_rect() +
```

```
#facet_wrap(~chrom, scales = "free_x", ncol = 3) +
scale_fill_manual(values=custom_colors, labels=c("CUR/ARU", "BEL/MX", "DRT/FLL/FLU", "JAM/DOM", "PAN"))
labs(title = "Runs of Homozygosity Chromosome 4",
     x = "Position (Mb)",
     fill = "Population") +
theme_minimal(base_size = 12) +
theme(axis.text.y = element_text(size = 8),
      axis.ticks.x = element_blank(),
      strip.background = element_rect(fill = "grey90", color = NA),
      strip.text = element_text(face = "bold"))
```

Runs of Homozygosity Chromosome 4

